PROGRAMACIÓN III · ANGULAR + .NET

Paginación y Filtrado

Apunte completo de estudio

Query parameters · LINQ Skip/Take · HttpParams
Filtrado múltiple · Debounce con RxJS

Instituto Técnico Superior Córdoba

Programación III · Prof. Mateo Alexis Vranich Prieto · Córdoba 2026

#	Sección
01	¿Por qué paginar?
02	Query parameters: el mecanismo de comunicación
03	Backend: paginación con .NET y LINQ
04	Backend: el controlador
05	Frontend: modelo e interfaz en Angular
06	Frontend: el servicio con HttpParams
07	Frontend: el componente y el template

01 · ¿Por qué paginar?

Hasta ahora, cada vez que pedimos datos a la API usamos endpoints como `GET /api/productos` que retornan **todos** los registros de la tabla. Esto funciona con 10 o 50 registros, pero ¿qué pasa cuando la tabla tiene 10.000 productos? El servidor genera un JSON enorme, lo transmite por la red, el navegador lo recibe, Angular lo parsea y el DOM intenta renderizar 10.000 filas. El resultado: una app lenta, un servidor sobrecargado y una experiencia de usuario terrible.

La **paginación** resuelve esto dividiendo los resultados en páginas más pequeñas. En lugar de pedir todos los registros, pedimos "dame los registros del 21 al 30" — una página de 10 elementos. El servidor consulta solo esos registros en la base de datos, transmite un JSON pequeño, y Angular renderiza solo 10 filas. Todos ganan.

> **Analogía del libro:** Un libro no muestra todo su contenido en una sola hoja. Lo divide en páginas que podés recorrer una por una. Nuestra API hace exactamente lo mismo: en lugar de entregar todos los datos de golpe, los entrega página por página. El cliente dice "quiero la página 3 con 10 elementos por página" y el servidor responde solo con esos 10 elementos.

El **filtrado** complementa a la paginación: permite que el usuario busque dentro de los datos. "Mostrame solo los productos que contengan *notebook* en el nombre" o "solo los de la categoría *Electrónica*". El filtro se aplica **antes** de paginar, así la paginación opera sobre el subconjunto filtrado.

> **Punto en común con el backend:** en el apunte de LINQ ya vieron `Skip()` y `Take()` para paginación, y `Where()` para filtrado. Ahora vamos a ver cómo el frontend le indica al backend qué página y qué filtro quiere, usando query parameters en la URL.

02 · Query parameters: el mecanismo de comunicación

Los query parameters son pares clave-valor que van al final de una URL, después del signo `?`, separados por `&`. No forman parte de la ruta del recurso — son información adicional que modifica el comportamiento de la consulta.

Estructura de una URL con query parameters

```
GET /api/productos?page=2&pageSize=10&search=notebook&category=Electrónica
    └── recurso ──┘ └──────────── query parameters ──────────────┘

// Cada query parameter es un par clave=valor:
//   page=2         → número de página solicitada
//   pageSize=10    → cantidad de elementos por página
//   search=notebook → texto de búsqueda
//   category=...    → filtro por categoría
```

Los query parameters son la forma estándar de enviar opciones de consulta en APIs REST. No modifican el recurso (siguen siendo "productos"), solo modifican **cómo** se devuelven. Son opcionales: si el cliente no los envía, la API usa valores por defecto.

> **Punto en común con el backend:** en .NET, los query parameters se reciben con el atributo `[FromQuery]` en los parámetros del controlador. Angular los envía usando el objeto `HttpParams` del módulo HttpClient.

03 · Backend: paginación con .NET y LINQ

DTO de respuesta paginada

El primer paso es definir un DTO (Data Transfer Object) genérico que encapsule la respuesta paginada. No solo devolvemos la lista de elementos, sino también la información de paginación que el frontend necesita para renderizar los controles (página actual, total de páginas, etc.).

DTOs/PagedResponse.cs

```
public class PagedResponse<T>
{
    public List<T> Items { get; set; }     // los elementos de esta página
    public int Page { get; set; }          // página actual
    public int PageSize { get; set; }      // tamaño de página
    public int TotalCount { get; set; }    // total de registros (sin paginar)
    public int TotalPages { get; set; }    // total de páginas
    public bool HasPrevious => Page > 1;
    public bool HasNext => Page < TotalPages;
}
```

> **¿Por qué genérico?** Usamos `PagedResponse<T>` con un tipo genérico para poder reutilizarlo con cualquier entidad: PagedResponse<Producto>, PagedResponse<Cliente>, PagedResponse<Pedido>. Una sola clase para todas las entidades paginadas.

El servicio con paginación y filtrado

En el servicio es donde aplicamos la lógica de LINQ. El orden es importante: primero filtramos con `Where()`, después contamos el total con `CountAsync()`, y finalmente paginamos con `Skip()` y `Take()`.

Services/ProductoService.cs

```
public async Task<PagedResponse<Producto>> ObtenerPaginadoAsync(
    int page = 1, int pageSize = 10, string? search = null)
{
    // 1. Empezamos con la consulta base (IQueryable)
    var query = _db.Productos.AsQueryable();

    // 2. FILTRAR — aplicamos el Where ANTES de paginar
    if (!string.IsNullOrWhiteSpace(search))
    {
        query = query.Where(p =>
            p.Nombre.Contains(search) ||
            p.Descripcion.Contains(search));
    }

    // 3. CONTAR — total de registros que coinciden con el filtro
    var totalCount = await query.CountAsync();

    // 4. PAGINAR — Skip salta los anteriores, Take toma la página
    var items = await query
        .OrderBy(p => p.Nombre)
        .Skip((page - 1) * pageSize)   // si page=2, skip=10
        .Take(pageSize)                 // toma 10 elementos
        .ToListAsync();

    // 5. ARMAR la respuesta paginada
    return new PagedResponse<Producto>
    {
        Items = items,
        Page = page,
        PageSize = pageSize,
        TotalCount = totalCount,
        TotalPages = (int)Math.Ceiling(totalCount / (double)pageSize)
    };
}
```

Desglosemos la fórmula de `Skip()`. Si queremos la página 2 con 10 elementos por página, necesitamos saltear los primeros 10:

page	pageSize	(page - 1) * pageSize	Skip	Take	Registros
1	10	(1-1) * 10 = 0	0	10	1 al 10
2	10	(2-1) * 10 = 10	10	10	11 al 20
3	10	(3-1) * 10 = 20	20	10	21 al 30
5	20	(5-1) * 20 = 80	80	20	81 al 100

> **IQueryable vs IEnumerable:** Usamos `AsQueryable()` para que los filtros y la paginación se traduzcan a SQL y se ejecuten en la base de datos. Si usáramos `ToList()` antes del `Where()`, primero traería TODOS los registros a memoria y después filtraría en C# — exactamente lo que queremos evitar.

04 · Backend: el controlador

El controlador recibe los query parameters con `[FromQuery]` y se los pasa al servicio. Notar que los parámetros tienen valores por defecto: si el cliente no envía `page`, el controlador usa 1.

Controllers/ProductosController.cs

```
[ApiController]
[Route("api/[controller]")]
public class ProductosController : ControllerBase
{
    private readonly ProductoService _service;
    public ProductosController(ProductoService service)
        => _service = service;

    [HttpGet]
    public async Task<IActionResult> GetAll(
        [FromQuery] int page = 1,
        [FromQuery] int pageSize = 10,
        [FromQuery] string? search = null)
    {
        // Validar límites
        if (page < 1) page = 1;
        if (pageSize < 1) pageSize = 10;
        if (pageSize > 50) pageSize = 50; // límite máximo

        var result = await _service.ObtenerPaginadoAsync(
            page, pageSize, search);
        return Ok(result);
    }
}
```

Ahora la API responde con un JSON que incluye tanto los datos como la metadata de paginación:

Response · GET /api/productos?page=2&pageSize=5&search=teclado

```
{
  "items": [
    { "id": 6, "nombre": "Teclado mecánico", "precio": 8500 },
    { "id": 11, "nombre": "Teclado inalámbrico", "precio": 4200 }
  ],
  "page": 2,
  "pageSize": 5,
  "totalCount": 7,
  "totalPages": 2,
  "hasPrevious": true,
  "hasNext": false
}
```

> **Punto en común con el backend:** fijate cómo el controlador solo recibe, valida y delega. Toda la lógica de filtrado y paginación está en el servicio — el mismo patrón de capas (Controlador → Servicio → Repositorio) que venimos usando.

05 · Frontend: modelo e interfaz en Angular

Del lado de Angular, necesitamos un modelo que espeje la respuesta paginada del backend. Usamos una interfaz genérica que coincida con la estructura del `PagedResponse<T>` de .NET:

models/paged-response.model.ts

```
export interface PagedResponse<T> {
    items: T[];             // los elementos de esta página
    page: number;           // página actual
    pageSize: number;       // tamaño de página
    totalCount: number;     // total de registros
    totalPages: number;     // total de páginas
    hasPrevious: boolean;   // ¿hay página anterior?
    hasNext: boolean;       // ¿hay página siguiente?
}
```

> **Espejo frontend-backend:** Observá cómo la interfaz de Angular es un espejo exacto de la clase de .NET. Los nombres de las propiedades coinciden (camelCase en Angular, PascalCase en .NET — la serialización JSON de .NET convierte automáticamente a camelCase). Este patrón de "clase en C# → interfaz en TypeScript" lo vimos con LoginResponse en el apunte de autenticación.

06 · Frontend: el servicio con HttpParams

El servicio de Angular debe enviar los query parameters al backend. Para eso usamos `HttpParams`, una clase del módulo `@angular/common/http` que construye los pares clave-valor de forma segura y tipada.

services/producto.service.ts

```
import { Injectable } from '@angular/core';
import { HttpClient, HttpParams } from '@angular/common/http';
import { Observable } from 'rxjs';
import { environment } from '../environments/environment';
import { PagedResponse } from '../models/paged-response.model';
import { Producto } from '../models/producto.model';

@Injectable({ providedIn: 'root' })
export class ProductoService {
    private apiUrl = environment.apiUrl + '/productos';
    constructor(private http: HttpClient) {}

    getAll(page: number = 1, pageSize: number = 10,
           search: string = ''): Observable<PagedResponse<Producto>> {
        // Construir los query parameters
        let params = new HttpParams()
            .set('page', page.toString())
            .set('pageSize', pageSize.toString());

        // Solo agregar search si tiene valor
        if (search.trim()) {
            params = params.set('search', search.trim());
        }

        // El segundo argumento de get() acepta { params }
        return this.http.get<PagedResponse<Producto>>(this.apiUrl, { params });
    }
}
```

`HttpParams` es inmutable: cada llamada a `.set()` retorna un nuevo objeto. Por eso usamos `params = params.set(...)` y no solo `params.set(...)`. Esto es el mismo concepto de inmutabilidad que vimos con `req.clone()` en los interceptors.

Cuando Angular ejecuta este request, la URL final que sale del navegador es:

URL final

```
GET http://localhost:5000/api/productos?page=2&pageSize=10&search=notebook
                                        └──── HttpParams los agregó ────┘
```

07 · Frontend: el componente y el template

El componente es el que conecta todo: tiene el campo de búsqueda, muestra la tabla de productos y los botones de paginación. Cada vez que el usuario cambia de página o escribe en el buscador, el componente llama al servicio con los nuevos parámetros.

components/productos/productos.component.ts

```
@Component({
    standalone: true,
    selector: 'app-productos',
    imports: [CommonModule, FormsModule],
    templateUrl: './productos.component.html'
})
export class ProductosComponent implements OnInit {
    productos: Producto[] = [];
    page = 1;
    pageSize = 10;
    totalPages = 0;
    totalCount = 0;
    search = '';

    constructor(private productoService: ProductoService) {}

    ngOnInit(): void { this.loadProductos(); }

    loadProductos(): void {
        this.productoService.getAll(this.page, this.pageSize, this.search)
            .subscribe({
                next: (response: PagedResponse<Producto>) => {
                    this.productos = response.items;
                    this.page = response.page;
                    this.totalPages = response.totalPages;
                    this.totalCount = response.totalCount;
                },
                error: (err) => console.error('Error:', err)
            });
    }

    goToPage(p: number): void {
        if (p >= 1 && p <= this.totalPages) {
            this.page = p;
            this.loadProductos();
        }
    }

    onSearch(): void {
        this.page = 1; // siempre volver a la primera página
        this.loadProductos();
    }
}
```

El template HTML

productos.component.html

```
<div class="container">
  <!-- Barra de búsqueda -->
  <div class="search-bar">
    <input type="text" [(ngModel)]="search"
           placeholder="Buscar productos..."
           (keyup.enter)="onSearch()" />
    <button (click)="onSearch()">Buscar</button>
  </div>

  <p>Mostrando página {{ page }} de {{ totalPages }}
     ({{ totalCount }} resultados)</p>

  <table>
    <thead><tr><th>ID</th><th>Nombre</th><th>Precio</th></tr></thead>
    <tbody>
      <tr *ngFor="let p of productos">
        <td>{{ p.id }}</td>
        <td>{{ p.nombre }}</td>
        <td>{{ p.precio | currency:'ARS' }}</td>
      </tr>
    </tbody>
  </table>

  <!-- Controles de paginación -->
  <div class="pagination">
    <button (click)="goToPage(page - 1)" [disabled]="page <= 1">Anterior</button>
    <span *ngFor="let p of [].constructor(totalPages); let i = index">
      <button (click)="goToPage(i + 1)" [class.active]="page === i + 1">{{ i + 1 }}</button>
    </span>
    <button (click)="goToPage(page + 1)" [disabled]="page >= totalPages">Siguiente</button>
  </div>
</div>
```

> **¿Por qué resetear a página 1 al buscar?** Si el usuario está en la página 5 y busca "mouse", probablemente el resultado filtrado tenga menos de 5 páginas. Si no volvemos a la página 1, la API retornaría una página vacía. Por eso `onSearch()` siempre setea `this.page = 1` antes de recargar.

08 · Filtrado avanzado: múltiples filtros

En una aplicación real, es común necesitar más de un filtro. Por ejemplo: filtrar por nombre, por categoría y por rango de precio al mismo tiempo. El patrón es el mismo: cada filtro se agrega como un query parameter adicional.

Backend: servicio con múltiples filtros

Services/ProductoService.cs

```
public async Task<PagedResponse<Producto>> ObtenerPaginadoAsync(
    int page = 1, int pageSize = 10, string? search = null,
    int? categoriaId = null, decimal? precioMin = null, decimal? precioMax = null)
{
    var query = _db.Productos.Include(p => p.Categoria).AsQueryable();

    // Cada filtro se encadena con Where si tiene valor
    if (!string.IsNullOrWhiteSpace(search))
        query = query.Where(p => p.Nombre.Contains(search));
    if (categoriaId.HasValue)
        query = query.Where(p => p.CategoriaId == categoriaId);
    if (precioMin.HasValue)
        query = query.Where(p => p.Precio >= precioMin);
    if (precioMax.HasValue)
        query = query.Where(p => p.Precio <= precioMax);

    var totalCount = await query.CountAsync();
    var items = await query
        .OrderBy(p => p.Nombre)
        .Skip((page - 1) * pageSize).Take(pageSize)
        .ToListAsync();

    return new PagedResponse<Producto> { /* ... */ };
}
```

Observá el patrón: cada filtro es un `if` que **encadena** un `Where()` adicional sobre la misma variable `query`. Si el parámetro es null, ese filtro simplemente no se aplica. LINQ traduce todos los `Where()` encadenados a una sola sentencia SQL con múltiples condiciones AND.

Frontend: servicio con múltiples filtros

services/producto.service.ts

```
getAll(page = 1, pageSize = 10, search = '',
       categoriaId: number | null = null,
       precioMin: number | null = null,
       precioMax: number | null = null): Observable<PagedResponse<Producto>> {

    let params = new HttpParams()
        .set('page', page.toString())
        .set('pageSize', pageSize.toString());

    if (search.trim())     params = params.set('search', search.trim());
    if (categoriaId !== null) params = params.set('categoriaId', categoriaId.toString());
    if (precioMin !== null) params = params.set('precioMin', precioMin.toString());
    if (precioMax !== null) params = params.set('precioMax', precioMax.toString());

    return this.http.get<PagedResponse<Producto>>(this.apiUrl, { params });
}
```

> **Patrón espejo:** fijate cómo el servicio de Angular replica exactamente la misma lógica del servicio de .NET: "si el filtro tiene valor, lo agrego". En el backend se encadena un `Where()`, en el frontend se encadena un `params.set()`. La simetría hace que ambas capas sean fáciles de mantener.

09 · Flujo completo y comparativo

Veamos el recorrido completo de un request paginado con filtro, desde que el usuario escribe en el buscador hasta que ve los resultados en pantalla:

Paso	Capa	Qué sucede
1	**Usuario**	Escribe "notebook" en el buscador y presiona Enter
2	**Componente**	Ejecuta onSearch(): setea page=1 y llama a loadProductos()
3	**Servicio Angular**	Construye HttpParams: page=1, pageSize=10, search=notebook
4	**Interceptor**	Agrega el header Authorization: Bearer
5	**Navegador**	Envía GET /api/productos?page=1&pageSize=10&search=notebook
6	**API .NET**	UseAuthentication valida el JWT
7	**Controlador**	Recibe los [FromQuery] params y los pasa al servicio
8	**Servicio .NET**	Aplica Where(Contains("notebook")), cuenta, Skip/Take
9	**EF Core / LINQ**	Traduce a SQL: WHERE Nombre LIKE '%notebook%' OFFSET 0 FETCH 10
10	**Servicio .NET**	Arma el PagedResponse con items + metadata
11	**Controlador**	Retorna Ok(result) → 200 con el JSON paginado
12	**Componente**	Recibe el response, actualiza productos[], page, totalPages
13	**Template**	Renderiza la tabla con 10 filas y los botones de paginación

Comparativo Backend vs Frontend

Responsabilidad	.NET (Backend)	Angular (Frontend)
Recibir parámetros	[FromQuery] int page	HttpParams.set("page", ...)
Filtrar datos	query.Where(p => ...)	params.set("search", ...)
Paginar datos	query.Skip().Take()	El backend lo hace
Contar total	query.CountAsync()	Lee totalCount del response
Estructura de respuesta	PagedResponse (clase C#)	PagedResponse (interfaz TS)
Controles de navegación	Calcula HasPrevious/HasNext	Renderiza botones con *ngFor
Ejecutar búsqueda	Contains() → SQL LIKE	onSearch() → nueva llamada HTTP
Resetear al buscar	No aplica	page = 1 antes de recargar

10 · Debounce: optimizar la búsqueda en tiempo real

Si quisiéramos que la búsqueda se ejecute mientras el usuario escribe (sin presionar Enter), tendríamos un problema: cada tecla dispararía un request HTTP. Si el usuario escribe "notebook", serían 8 requests en menos de un segundo. Esto sobrecarga la API innecesariamente.

La técnica de **debounce** resuelve esto: espera un breve tiempo (por ejemplo 300ms) después de la última tecla antes de ejecutar el request. Si el usuario sigue escribiendo, el timer se reinicia. Solo cuando deja de escribir por 300ms se dispara la búsqueda.

Debounce con RxJS (en el componente)

```
import { Subject, debounceTime, distinctUntilChanged } from 'rxjs';

export class ProductosComponent implements OnInit, OnDestroy {
    private searchSubject = new Subject<string>();

    ngOnInit(): void {
        // Configurar el debounce
        this.searchSubject.pipe(
            debounceTime(300),        // esperar 300ms
            distinctUntilChanged()    // ignorar si no cambió
        ).subscribe(term => {
            this.search = term;
            this.page = 1;
            this.loadProductos();
        });
        this.loadProductos();
    }

    // El input llama a este método en cada tecla
    onSearchChange(term: string): void {
        this.searchSubject.next(term);
    }

    ngOnDestroy(): void { this.searchSubject.complete(); }
}
```

template · usar (input) en lugar de (keyup.enter)

```
<input type="text" [ngModel]="search"
       (input)="onSearchChange($event.target.value)"
       placeholder="Buscar productos..." />
```

> **Analogía del ascensor:** El debounce funciona como la puerta de un ascensor: no se cierra inmediatamente cuando apretás el botón. Espera unos segundos por si alguien más quiere subir. Si llega alguien, el timer se reinicia. Solo cuando nadie más llega, la puerta se cierra y el ascensor arranca. Cada tecla del usuario es "alguien llegando" — el request solo sale cuando nadie más teclea.

11 · Errores comunes y resumen

Errores comunes

Error	Causa	Solución
La primera página muestra datos pero las siguientes están vacías	Se aplica ToList() antes de Skip/Take, trayendo todo a memoria primero	Asegurarse de que Skip/Take se apliquen sobre IQueryable, no sobre una lista ya materializada
Al buscar, la tabla queda vacía aunque hay resultados	No se resetea page a 1 al cambiar el filtro	Setear this.page = 1 antes de llamar a loadProductos() en onSearch()
El filtro busca de forma case-sensitive y no encuentra resultados	Contains() en EF Core depende del collation de la BD	Usar ToLower() en ambos lados: p.Nombre.ToLower().Contains(search.ToLower())
La API recibe null en los parámetros	HttpParams envía strings pero la API espera int	Usar .toString() al setear cada param en Angular
Demasiados requests al escribir en el buscador	Cada tecla dispara un request HTTP	Implementar debounce con RxJS (sección 10)

Resumen de conceptos clave

Concepto	Definición rápida
Paginación	Dividir los resultados en páginas de tamaño fijo para no cargar todos los datos de una vez
Filtrado	Reducir los resultados aplicando condiciones de búsqueda antes de paginar
Query parameter	Par clave=valor al final de la URL (?page=2&search=x) para enviar opciones de consulta
[FromQuery]	Atributo de .NET que indica que un parámetro del controlador viene de los query params de la URL
HttpParams	Clase de Angular para construir query parameters de forma inmutable y tipada
Skip(n)	Método LINQ que salta los primeros n registros. Fórmula: (page - 1) * pageSize
Take(n)	Método LINQ que toma los siguientes n registros (el tamaño de página)

Concepto	Definición rápida
IQueryable	Tipo que permite encadenar operaciones LINQ que se traducen a SQL sin ejecutarse hasta el final
PagedResponse	DTO genérico que contiene los items + metadata de paginación (página, total, etc.)
Debounce	Técnica que espera un tiempo tras la última acción del usuario antes de ejecutar la operación
distinctUntilChanged	Operador RxJS que ignora emisiones si el valor no cambió respecto al anterior
totalPages	Se calcula con Math.Ceiling(totalCount / pageSize) — redondeo hacia arriba